Learner Guide

Python Text Mining and Analytics: Transforming Text into Insights

TGS-2026064715 · WSQ · 2 days / 16 hours · v15 · 14 September 2026

Tertiary Infotech Academy Pte Ltd · UEN 201200696W

© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

Document Version Control Record

Version	Release date	Change	Owner
v14	Previous release	Legacy courseware	Tertiary Infotech Academy
v15	14 September 2026	Corrected course code; rebuilt visual slides; aligned synthetic-data labs, LG and LP	Tertiary Infotech Academy

© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

TABLE OF CONTENTS

Document Version Control Record....4
Learning outcomes....7
Environment and data....7
Topic 1: CRISP-DM and corpus design....7
Business question....7
CRISP-DM phases....7
Data inventory....7
Label schema....7
Corpus provenance....7
Sampling frame....7
Success metric....7
Train-test boundary....8
Baseline analysis....8
Deployment evidence....8
NLU and NLG boundary....8
Topic 2: Text cleaning and vectorization....8
Corpus ingestion....8
Unicode normalization....8
Tokenization....8
Case folding....8
Punctuation policy....8
Stop-word policy....8
Stemming....8
Lemmatization....9
Term frequency....9
TF-IDF and n-grams....9
Topic 3: Text analytics and feature engineering....9
POS tags....9
Named entities....9
Entity normalization....9
Co-occurrence links....9
Aspect extraction....9
Feature engineering....9
K-means clustering....9
Elbow diagnostic....9
Supervised classification....10
Error analysis....10
Topic 4: Sentiment modelling and evaluation....10

© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

Lexicon baseline....10
Sentiment labels....10
Stratified split....10
Pipeline isolation....10
Model fit....10
Predictions....10
Confusion matrix....10
Precision and recall....10
Macro F1....10
Model card....11
TextBlob polarity....11
Topic 5: Summarization and visualization....11
Aspect sentiment....11
Hotel comparison....11
Trend analysis....11
Extractive summary....11
Topic modelling....11
LDA vs NMF....11
Visual encoding....11
Insight statement....11
Quality check....11
Delivery pack....12
Transformer summarization....12
Extractive vs abstractive....12
Lab 01: Business Question....12
Detailed procedure....12
Code walkthrough and interpretation....13
Lab 02: Corpus Ingestion....13
Detailed procedure....13
Code walkthrough and interpretation....13
Lab 03: Clean Tokenize....14
Detailed procedure....14
Code walkthrough and interpretation....14
Lab 04: Stemming Lemmatization....15
Detailed procedure....15
Code walkthrough and interpretation....15
Lab 05: Tf Ngram....16
Detailed procedure....16
Code walkthrough and interpretation....16

© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

Lab 06: Tfidf **17**
Detailed procedure 17
Code walkthrough and interpretation 17
Lab 07: Pos Ner Links **18**
Detailed procedure 18
Code walkthrough and interpretation 18
Lab 08: Cluster Classify **19**
Detailed procedure 20
Code walkthrough and interpretation 20
Lab 09: Sentiment Model **20**
Detailed procedure 20
Code walkthrough and interpretation 20
Lab 10: Summarize Visualize **21**
Detailed procedure 21
Code walkthrough and interpretation 21
Assessment preparation **22**

© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

Learning outcomes

- Design a text analytics solution using CRISP-DM.
- Read and preprocess a text corpus in Python.
- Engineer features and perform text analytics.
- Build and evaluate a sentiment model.
- Summarize and visualize text insights.

Environment and data

Use Python 3.10 or later. The 10 lab folders contain self-contained synthetic hotel-review CSVs and runnable scripts. No external API keys or customer data are needed. Run each script from its own folder. Install the shared Python requirements with python3 -m pip install -r labs/requirements.txt.

Topic 1: CRISP-DM and corpus design

Business question

Artifact: negative share by hotel. Analytical decision: define target, unit and decision. Check: sampling bias. Code/example: review_id + sentiment.

CRISP-DM phases

Artifact: six linked phases. Analytical decision: iterate evaluation into preparation. Check: linear waterfall assumption. Code/example: business → data → prepare → model → evaluate → deploy.

Data inventory

Artifact: review_id, date, hotel, text. Analytical decision: profile rows and nulls. Check: duplicate IDs. Code/example: df.info(); df.isna().sum().

Label schema

Artifact: positive / neutral / negative. Analytical decision: define annotation rules. Check: rating-label disagreement. Code/example: df.sentiment.value_counts().

Corpus provenance

Artifact: synthetic hotel reviews. Analytical decision: retain source and date fields. Check: unlicensed scraped text. Code/example: review_id, channel, date.

Sampling frame

Artifact: hotel × channel × month. Analytical decision: compare strata proportions. Check: channel overrepresentation. Code/example: pd.crosstab(df.hotel, df.channel).

Success metric

Artifact: macro F1 and negative recall. Analytical decision: tie metric to response triage. Check: accuracy hides minority failure. Code/example: classification_report(y_true, y_pred).

© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

Train-test boundary

Artifact: stratified holdout. Analytical decision: fit vocabulary on train only. Check: TF-IDF leakage. Code/example: train_test_split(..., stratify=y).

Baseline analysis

Artifact: majority and keyword rules. Analytical decision: compare value above baseline. Check: overclaiming uplift. Code/example: DummyClassifier(strategy="most_frequent").

Deployment evidence

Artifact: weekly insight dashboard. Analytical decision: specify owner and drift monitor. Check: model decay. Code/example: review volume + macro F1 trend.

NLU and NLG boundary

Artifact: intent extraction vs text generation. Analytical decision: choose task-specific output. Check: generated text mistaken for evidence. Code/example: NLU: classify; NLG: compose.

Topic 2: Text cleaning and vectorization

Corpus ingestion

Artifact: CSV to DataFrame. Analytical decision: validate encoding and schema. Check: silent dtype coercion. Code/example: pd.read_csv("hotel_reviews.csv").

Unicode normalization

Artifact: NFKC form. Analytical decision: compare before/after tokens. Check: meaningful symbols lost. Code/example: unicodedata.normalize("NFKC", text).

Tokenization

Artifact: regex word tokens. Analytical decision: measure token count. Check: contractions split. Code/example: re.findall(r"[A-Za-z]+", text).

Case folding

Artifact: lowercase vocabulary. Analytical decision: collapse variant terms. Check: acronym loss. Code/example: text.lower().

Punctuation policy

Artifact: preserve useful punctuation. Analytical decision: compare sentiment cue retention. Check: negation lost. Code/example: re.sub(r"[^\w\s]", " ", text).

Stop-word policy

Artifact: domain stop list. Analytical decision: retain not/no for sentiment. Check: negation removal. Code/example: CountVectorizer(stop_words=None).

Stemming

Artifact: Porter stems. Analytical decision: compare stem collisions. Check: interpretability loss. Code/example: PorterStemmer().stem("booking").

© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

Lemmatization

Artifact: WordNet lemmas. Analytical decision: inspect part-of-speech dependence. Check: missing WordNet corpus. Code/example: WordNetLemmatizer().lemmatize("rooms").

Term frequency

Artifact: document-term matrix. Analytical decision: inspect vocabulary dimensions. Check: rare-term noise. Code/example: CountVectorizer(min_df=2).

TF-IDF and n-grams

Artifact: weighted sparse matrix. Analytical decision: compare bigram signal. Check: high-dimensional sparsity. Code/example: TfidfVectorizer(ngram_range=(1,2)).

Topic 3: Text analytics and feature engineering

POS tags

Artifact: tokens with grammatical roles. Analytical decision: identify issue noun phrases. Check: tagger domain drift. Code/example: nltk.pos_tag(tokens).

Named entities

Artifact: hotel and location spans. Analytical decision: measure entity precision. Check: brand aliases missed. Code/example: entity = hotel_name.

Entity normalization

Artifact: canonical hotel names. Analytical decision: merge spelling variants. Check: false merges. Code/example: alias_map.get(name,name).

Co-occurrence links

Artifact: hotel × aspect edges. Analytical decision: count review-level links. Check: long documents inflate links. Code/example: links[(hotel, aspect)] += 1.

Aspect extraction

Artifact: booking / room / support. Analytical decision: inspect phrase context. Check: ambiguous terms. Code/example: text.str.contains("booking").

Feature engineering

Artifact: rating, channel, length. Analytical decision: compare incremental value. Check: label leakage via rating. Code/example: ColumnTransformer(...).

K-means clustering

Artifact: TF-IDF document vectors. Analytical decision: inspect inertia curve. Check: cluster labels mistaken for truth. Code/example: KMeans(n_clusters=3).

Elbow diagnostic

Artifact: inertia for k=2..5. Analytical decision: look for diminishing returns. Check: no clear elbow. Code/example: model.inertia_.

© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

Supervised classification

Artifact: labeled review corpus. Analytical decision: compare macro F1. Check: class imbalance. Code/example: LogisticRegression(max_iter=1000).

Error analysis

Artifact: false negative examples. Analytical decision: review mistakes by aspect. Check: metric-only evaluation. Code/example: confusion_matrix(y_true,y_pred).

Topic 4: Sentiment modelling and evaluation

Lexicon baseline

Artifact: positive/negative term scores. Analytical decision: inspect negation examples. Check: domain jargon. Code/example: score = pos_hits - neg_hits.

Sentiment labels

Artifact: three-class annotation. Analytical decision: audit rating mismatch. Check: weak labels. Code/example: pd.crosstab(rating,sentiment).

Stratified split

Artifact: 75/25 train/holdout. Analytical decision: preserve class proportions. Check: tiny holdout. Code/example: train_test_split(..., stratify=y).

Pipeline isolation

Artifact: TF-IDF then logistic model. Analytical decision: fit on training fold only. Check: pre-fit vectorizer leakage. Code/example: make_pipeline(TfidfVectorizer(), LogisticRegression()).

Model fit

Artifact: regularized logistic regression. Analytical decision: inspect coefficients. Check: overfitting. Code/example: pipe.fit(X_train,y_train).

Predictions

Artifact: class and confidence. Analytical decision: sample uncertain cases. Check: confidence calibration. Code/example: pipe.predict(X_test).

Confusion matrix

Artifact: true × predicted counts. Analytical decision: inspect negative misses. Check: axis swap. Code/example: confusion_matrix(y_test,pred).

Precision and recall

Artifact: per-class metrics. Analytical decision: trade triage capacity vs misses. Check: accuracy-only reporting. Code/example: classification_report(y_test,pred).

Macro F1

Artifact: unweighted class mean. Analytical decision: compare to majority baseline. Check: support ignored. Code/example: f1_score(...,average="macro").

© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

Model card

Artifact: data, metrics, limits. Analytical decision: document deployment threshold. Check: unsupported claims. Code/example: model_version + training_date.

TextBlob polarity

Artifact: lexicon-derived polarity score. Analytical decision: compare to supervised holdout. Check: domain-specific wording. Code/example: TextBlob(text).sentiment.polarity.

Topic 5: Summarization and visualization

Aspect sentiment

Artifact: aspect × polarity table. Analytical decision: rank negative issues. Check: mention overlap. Code/example: pd.crosstab(aspect,sentiment).

Hotel comparison

Artifact: normalized sentiment shares. Analytical decision: compare property mix. Check: different sample sizes. Code/example: pd.crosstab(hotel,sentiment,normalize="index").

Trend analysis

Artifact: monthly negative share. Analytical decision: detect sustained change. Check: small cell counts. Code/example: groupby(month).mean().

Extractive summary

Artifact: representative review sentences. Analytical decision: preserve source ID. Check: misleading cherry-pick. Code/example: rank by cosine similarity.

Topic modelling

Artifact: document-topic distribution. Analytical decision: inspect top terms. Check: unstable topics. Code/example: LatentDirichletAllocation(...).

LDA vs NMF

Artifact: probabilistic vs matrix factorization. Analytical decision: compare topic coherence. Check: topic count arbitrary. Code/example: NMF(n_components=3).

Visual encoding

Artifact: stacked sentiment bars. Analytical decision: show denominators. Check: colour-only meaning. Code/example: ax.bar(..., label=...).

Insight statement

Artifact: observation + evidence + action. Analytical decision: tie finding to hotel owner. Check: causality overclaim. Code/example: negative share and count.

Quality check

Artifact: spot-check source reviews. Analytical decision: confirm summary fidelity. Check: hallucinated summary. Code/example: review_id trace.

© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

Delivery pack

Artifact: chart, table, methods. Analytical decision: reproduce with script and CSV. Check: unreproducible dashboard. Code/example: output/sentiment_by_hotel.csv.

Transformer summarization

Artifact: source text to condensed output. Analytical decision: compare summary with review IDs. Check: unsupported generated claim. Code/example: pipeline("summarization")(text).

Extractive vs abstractive

Artifact: selected sentences vs generated paraphrase. Analytical decision: verify factual consistency. Check: missing complaint qualifier. Code/example: source_sentence_id + summary.

Lab 01: Business Question

Course: TGS-2026064715 · **Maps to:** K1 K2; A1 A3

Produce a measurable problem statement and a corpus audit.

`hotel_reviews.csv` contains 120 fictional Singapore hotel and service reviews with dates, channel, rating, sentiment and property. Names and review text are synthetic; no customer records or external credentials are used.

1. Open a terminal in this lab folder. Confirm Python 3.10+ with `python3 --version`.

2. Install the shared requirements from the parent `labs/requirements.txt` with `python3 -m pip install -r ../requirements.txt`.

3. Inspect the first rows and column names in `hotel_reviews.csv`. Check that `review_id` is unique and `sentiment` has three classes.

4. Run `python3 run.py` from this folder. Read the printed output and any files created in `output/`.

5. Explain how the result supports the lab goal, what a production dataset would add, and one failure or bias risk.

The script exits with code 0. Verify that the reported row count is 120. For model labs, inspect the holdout scores rather than training accuracy; for summaries, inspect the exported CSV/PNG.

How would different hotel review wording, label quality, or data leakage change this result?

Detailed procedure

1. Open lab-01-business-question/hotel_reviews.csv and verify its columns, 120 synthetic rows, unique review IDs and sentiment labels.
2. From a terminal, change to labs/lab-01-business-question and run python3 run.py.
3. Read the output, and inspect any generated files in labs/lab-01-business-question/output/. Record the evidence in your notes.
4. Explain the result against the lab goal, then identify a sampling, label, preprocessing or modelling limitation.

© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

Code walkthrough and interpretation

Read this lab-specific block from run.py before executing it. Identify input variables, transformations, printed evidence and output files. Change one parameter at a time, rerun, then explain the difference.

```
if MODE=='profile':
    print('CRISP-DM: business understanding -> data understanding -> preparation -> modelling ->
evaluation -> deployment')
    print('Question: Which hotel/service aspects predict negative review share?')
    print(df.groupby('hotel')['sentiment'].value_counts(normalize=True).round(2))
```

Record the exact command, observed output, interpretation and one limitation. If a package is missing, install from labs/requirements.txt; if WordNet data is unavailable, the normalization lab prints a fallback vocabulary instead of downloading during class.

Lab 02: Corpus Ingestion

Course: TGS-2026064715 · **Maps to:** K3; A2

Detect missing values, duplicate IDs and class imbalance before modelling.

`hotel_reviews.csv` contains 120 fictional Singapore hotel and service reviews with dates, channel, rating, sentiment and property. Names and review text are synthetic; no customer records or external credentials are used.

1. Open a terminal in this lab folder. Confirm Python 3.10+ with `python3 --version`.

2. Install the shared requirements from the parent `labs/requirements.txt` with `python3 -m pip install -r ../requirements.txt`.

3. Inspect the first rows and column names in `hotel_reviews.csv`. Check that `review_id` is unique and `sentiment` has three classes.

4. Run `python3 run.py` from this folder. Read the printed output and any files created in `output/`.

5. Explain how the result supports the lab goal, what a production dataset would add, and one failure or bias risk.

The script exits with code 0. Verify that the reported row count is 120. For model labs, inspect the holdout scores rather than training accuracy; for summaries, inspect the exported CSV/PNG.

How would different hotel review wording, label quality, or data leakage change this result?

Detailed procedure

5. Open lab-02-corpus-ingestion/hotel_reviews.csv and verify its columns, 120 synthetic rows, unique review IDs and sentiment labels.
6. From a terminal, change to labs/lab-02-corpus-ingestion and run python3 run.py.
7. Read the output, and inspect any generated files in labs/lab-02-corpus-ingestion/output/. Record the evidence in your notes.
8. Explain the result against the lab goal, then identify a sampling, label, preprocessing or modelling limitation.

© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

Code walkthrough and interpretation

Read this lab-specific block from run.py before executing it. Identify input variables, transformations, printed evidence and output files. Change one parameter at a time, rerun, then explain the difference.

```
if MODE=='ingest':
    print('schema',df.dtypes.to_dict()); print('duplicates',df.review_id.duplicated().sum())
    print('class balance',df.sentiment.value_counts().to_dict())
    assert df.review_id.is_unique and df.text.notna().all()
```

Record the exact command, observed output, interpretation and one limitation. If a package is missing, install from labs/requirements.txt; if WordNet data is unavailable, the normalization lab prints a fallback vocabulary instead of downloading during class.

Lab 03: Clean Tokenize

Course: TGS-2026064715 · **Maps to:** K3 K4; A4

Compare lowercasing, punctuation removal and stop-word decisions on hotel reviews.

`hotel_reviews.csv` contains 120 fictional Singapore hotel and service reviews with dates, channel, rating, sentiment and property. Names and review text are synthetic; no customer records or external credentials are used.

1. Open a terminal in this lab folder. Confirm Python 3.10+ with `python3 --version`.

2. Install the shared requirements from the parent `labs/requirements.txt` with `python3 -m pip install -r ../requirements.txt`.

3. Inspect the first rows and column names in `hotel_reviews.csv`. Check that `review_id` is unique and `sentiment` has three classes.

4. Run `python3 run.py` from this folder. Read the printed output and any files created in `output/`.

5. Explain how the result supports the lab goal, what a production dataset would add, and one failure or bias risk.

The script exits with code 0. Verify that the reported row count is 120. For model labs, inspect the holdout scores rather than training accuracy; for summaries, inspect the exported CSV/PNG.

How would different hotel review wording, label quality, or data leakage change this result?

Detailed procedure

9. Open lab-03-clean-tokenize/hotel_reviews.csv and verify its columns, 120 synthetic rows, unique review IDs and sentiment labels.
10. From a terminal, change to labs/lab-03-clean-tokenize and run python3 run.py.
11. Read the output, and inspect any generated files in labs/lab-03-clean-tokenize/output/. Record the evidence in your notes.
12. Explain the result against the lab goal, then identify a sampling, label, preprocessing or modelling limitation.

© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

Code walkthrough and interpretation

Read this lab-specific block from run.py before executing it. Identify input variables, transformations, printed evidence and output files. Change one parameter at a time, rerun, then explain the difference.

```
if MODE=='tokens':
    stops={'the','and','was','my','a','in','to','on','at','is'}
    for t in texts.head(8):
        raw=re.findall(r"[A-Za-z']+",t.lower())
        print({'raw':raw,'filtered':[x for x in raw if x not in stops]})
```

Record the exact command, observed output, interpretation and one limitation. If a package is missing, install from labs/requirements.txt; if WordNet data is unavailable, the normalization lab prints a fallback vocabulary instead of downloading during class.

Lab 04: Stemming Lemmatization

Course: TGS-2026064715 · **Maps to:** K4; A4

Inspect Porter stems and lemmatized terms without hiding meaning loss.

`hotel_reviews.csv` contains 120 fictional Singapore hotel and service reviews with dates, channel, rating, sentiment and property. Names and review text are synthetic; no customer records or external credentials are used.

1. Open a terminal in this lab folder. Confirm Python 3.10+ with `python3 --version`.

2. Install the shared requirements from the parent `labs/requirements.txt` with `python3 -m pip install -r ../requirements.txt`.

3. Inspect the first rows and column names in `hotel_reviews.csv`. Check that `review_id` is unique and `sentiment` has three classes.

4. Run `python3 run.py` from this folder. Read the printed output and any files created in `output/`.

5. Explain how the result supports the lab goal, what a production dataset would add, and one failure or bias risk.

The script exits with code 0. Verify that the reported row count is 120. For model labs, inspect the holdout scores rather than training accuracy; for summaries, inspect the exported CSV/PNG.

How would different hotel review wording, label quality, or data leakage change this result?

Detailed procedure

13. Open lab-04-stemming-lemmatization/hotel_reviews.csv and verify its columns, 120 synthetic rows, unique review IDs and sentiment labels.
14. From a terminal, change to labs/lab-04-stemming-lemmatization and run python3 run.py.
15. Read the output, and inspect any generated files in labs/lab-04-stemming-lemmatization/output/. Record the evidence in your notes.
16. Explain the result against the lab goal, then identify a sampling, label, preprocessing or modelling limitation.

© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

Code walkthrough and interpretation

Read this lab-specific block from run.py before executing it. Identify input variables, transformations, printed evidence and output files. Change one parameter at a time, rerun, then explain the difference.

```
if MODE=='normalize':
    try:
        from nltk.stem import PorterStemmer, WordNetLemmatizer
        stem=PorterStemmer(); lem=WordNetLemmatizer()
        words=['booking','booked','rooms','delayed','service','services']
        print([(w,stem.stem(w),lem.lemmatize(w)) for w in words])
    except (ImportError, LookupError):
        print('Install optional nltk with: python -m pip install nltk')
        print('Fallback vocabulary:',sorted(set(re.findall(r'[a-z]+',' '.join(texts).lower())))[:30])
```

Record the exact command, observed output, interpretation and one limitation. If a package is missing, install from labs/requirements.txt; if WordNet data is unavailable, the normalization lab prints a fallback vocabulary instead of downloading during class.

Lab 05: Tf Ngram

Course: TGS-2026064715 · **Maps to:** K4; A4

Compare unigram and bigram vocabulary and term counts.

`hotel_reviews.csv` contains 120 fictional Singapore hotel and service reviews with dates, channel, rating, sentiment and property. Names and review text are synthetic; no customer records or external credentials are used.

1. Open a terminal in this lab folder. Confirm Python 3.10+ with `python3 --version`.

2. Install the shared requirements from the parent `labs/requirements.txt` with `python3 -m pip install -r ../requirements.txt`.

3. Inspect the first rows and column names in `hotel_reviews.csv`. Check that `review_id` is unique and `sentiment` has three classes.

4. Run `python3 run.py` from this folder. Read the printed output and any files created in `output/`.

5. Explain how the result supports the lab goal, what a production dataset would add, and one failure or bias risk.

The script exits with code 0. Verify that the reported row count is 120. For model labs, inspect the holdout scores rather than training accuracy; for summaries, inspect the exported CSV/PNG.

How would different hotel review wording, label quality, or data leakage change this result?

Detailed procedure

17. Open lab-05-tf-ngram/hotel_reviews.csv and verify its columns, 120 synthetic rows, unique review IDs and sentiment labels.
18. From a terminal, change to labs/lab-05-tf-ngram and run python3 run.py.
19. Read the output, and inspect any generated files in labs/lab-05-tf-ngram/output/. Record the evidence in your notes.

© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

20. Explain the result against the lab goal, then identify a sampling, label, preprocessing or modelling limitation.

Code walkthrough and interpretation

Read this lab-specific block from run.py before executing it. Identify input variables, transformations, printed evidence and output files. Change one parameter at a time, rerun, then explain the difference.

```
if MODE=='tf':
    for ngram in [(1,1),(1,2)]:
        v=CountVectorizer(stop_words='english',ngram_range=ngram,min_df=2)
        X=v.fit_transform(texts)
        top=sorted(zip(X.sum(axis=0).A1,v.get_feature_names_out()),reverse=True)[:12]
        print('ngram',ngram,'shape',X.shape,'top',top)
```

Record the exact command, observed output, interpretation and one limitation. If a package is missing, install from labs/requirements.txt; if WordNet data is unavailable, the normalization lab prints a fallback vocabulary instead of downloading during class.

Lab 06: Tfidf

Course: TGS-2026064715 · **Maps to:** K4; A4

Show why common terms receive lower inverse-document weight.

`hotel_reviews.csv` contains 120 fictional Singapore hotel and service reviews with dates, channel, rating, sentiment and property. Names and review text are synthetic; no customer records or external credentials are used.

1. Open a terminal in this lab folder. Confirm Python 3.10+ with `python3 --version`.

2. Install the shared requirements from the parent `labs/requirements.txt` with `python3 -m pip install -r ../requirements.txt`.

3. Inspect the first rows and column names in `hotel_reviews.csv`. Check that `review_id` is unique and `sentiment` has three classes.

4. Run `python3 run.py` from this folder. Read the printed output and any files created in `output/`.

5. Explain how the result supports the lab goal, what a production dataset would add, and one failure or bias risk.

The script exits with code 0. Verify that the reported row count is 120. For model labs, inspect the holdout scores rather than training accuracy; for summaries, inspect the exported CSV/PNG.

How would different hotel review wording, label quality, or data leakage change this result?

Detailed procedure

21. Open lab-06-tfidf/hotel_reviews.csv and verify its columns, 120 synthetic rows, unique review IDs and sentiment labels.
22. From a terminal, change to labs/lab-06-tfidf and run python3 run.py.
23. Read the output, and inspect any generated files in labs/lab-06-tfidf/output/. Record the evidence in your notes.

© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

24. Explain the result against the lab goal, then identify a sampling, label, preprocessing or modelling limitation.

Code walkthrough and interpretation

Read this lab-specific block from run.py before executing it. Identify input variables, transformations, printed evidence and output files. Change one parameter at a time, rerun, then explain the difference.

```
if MODE=='tfidf':
    v=TfidfVectorizer(stop_words='english',min_df=2)
    X=v.fit_transform(texts)
    for i in [0,1,2]:
        pairs=sorted(zip(X[i].toarray().ravel(),v.get_feature_names_out()),reverse=True)[:8]
        print(df.review_id[i],pairs)
    print('IDF examples',[(w,round(v.idf_[v.vocabulary_[w]],3)) for w in ['room','booking'] if w in
v.vocabulary_])
```

Record the exact command, observed output, interpretation and one limitation. If a package is missing, install from labs/requirements.txt; if WordNet data is unavailable, the normalization lab prints a fallback vocabulary instead of downloading during class.

Lab 07: Pos Ner Links

Course: TGS-2026064715 · **Maps to:** K5 K6; A5

Find hotel/location mentions and co-occurring service issues.

`hotel_reviews.csv` contains 120 fictional Singapore hotel and service reviews with dates, channel, rating, sentiment and property. Names and review text are synthetic; no customer records or external credentials are used.

1. Open a terminal in this lab folder. Confirm Python 3.10+ with `python3 --version`.

2. Install the shared requirements from the parent `labs/requirements.txt` with `python3 -m pip install -r ../requirements.txt`.

3. Inspect the first rows and column names in `hotel_reviews.csv`. Check that `review_id` is unique and `sentiment` has three classes.

4. Run `python3 run.py` from this folder. Read the printed output and any files created in `output/`.

5. Explain how the result supports the lab goal, what a production dataset would add, and one failure or bias risk.

The script exits with code 0. Verify that the reported row count is 120. For model labs, inspect the holdout scores rather than training accuracy; for summaries, inspect the exported CSV/PNG.

How would different hotel review wording, label quality, or data leakage change this result?

Detailed procedure

25. Open lab-07-pos-ner-links/hotel_reviews.csv and verify its columns, 120 synthetic rows, unique review IDs and sentiment labels.
26. From a terminal, change to labs/lab-07-pos-ner-links and run python3 run.py.

© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

27. Read the output, and inspect any generated files in labs/lab-07-pos-ner-links/output/. Record the evidence in your notes.
28. Explain the result against the lab goal, then identify a sampling, label, preprocessing or modelling limitation.

Code walkthrough and interpretation

Read this lab-specific block from run.py before executing it. Identify input variables, transformations, printed evidence and output files. Change one parameter at a time, rerun, then explain the difference.

```
if MODE=='entities':
    hotels=['Harbour View Hotel','Orchard Central Hotel','Marina Bay Suites']
    aspects=['booking','room','breakfast','support','invoice','app']
    links=collections.Counter()
    for row in df.itertuples():
        for aspect in aspects:
            if aspect in row.text.lower(): links[(row.hotel,aspect)]+=1
    print('Entity-aspect links',links.most_common(15))
    (OUT/'links.json').write_text(json.dumps([{'hotel':h,'aspect':a,'count':n} for (h,a),n in
links.items()],indent=2))
```

Record the exact command, observed output, interpretation and one limitation. If a package is missing, install from labs/requirements.txt; if WordNet data is unavailable, the normalization lab prints a fallback vocabulary instead of downloading during class.

Lab 08: Cluster Classify

Course: TGS-2026064715 · **Maps to:** K5 K6 K7; A5 A6

Compare a K-means elbow trace with a supervised classification baseline.

`hotel_reviews.csv` contains 120 fictional Singapore hotel and service reviews with dates, channel, rating, sentiment and property. Names and review text are synthetic; no customer records or external credentials are used.

1. Open a terminal in this lab folder. Confirm Python 3.10+ with `python3 --version`.

2. Install the shared requirements from the parent `labs/requirements.txt` with `python3 -m pip install -r ../requirements.txt`.

3. Inspect the first rows and column names in `hotel_reviews.csv`. Check that `review_id` is unique and `sentiment` has three classes.

4. Run `python3 run.py` from this folder. Read the printed output and any files created in `output/`.

5. Explain how the result supports the lab goal, what a production dataset would add, and one failure or bias risk.

The script exits with code 0. Verify that the reported row count is 120. For model labs, inspect the holdout scores rather than training accuracy; for summaries, inspect the exported CSV/PNG.

How would different hotel review wording, label quality, or data leakage change this result?

© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

Detailed procedure

29. Open lab-08-cluster-classify/hotel_reviews.csv and verify its columns, 120 synthetic rows, unique review IDs and sentiment labels.
30. From a terminal, change to labs/lab-08-cluster-classify and run python3 run.py.
31. Read the output, and inspect any generated files in labs/lab-08-cluster-classify/output/. Record the evidence in your notes.
32. Explain the result against the lab goal, then identify a sampling, label, preprocessing or modelling limitation.

Code walkthrough and interpretation

Read this lab-specific block from run.py before executing it. Identify input variables, transformations, printed evidence and output files. Change one parameter at a time, rerun, then explain the difference.

```
if MODE=='cluster':
    v=TfidfVectorizer(stop_words='english',min_df=2)
    X=v.fit_transform(texts)
    for k in [2,3,4,5]:
        m=KMeans(n_clusters=k,random_state=42,n_init=10).fit(X)
        print('k',k,'inertia',round(m.inertia_,2))
    pipe=make_pipeline(TfidfVectorizer(stop_words='english'),LogisticRegression(max_iter=1000))

Xtr,Xte,ytr,yte=train_test_split(texts,df.sentiment,test_size=.25,stratify=df.sentiment,random_state=42
)
    pipe.fit(Xtr,ytr); print(classification_report(yte,pipe.predict(Xte),zero_division=0))
```

Record the exact command, observed output, interpretation and one limitation. If a package is missing, install from labs/requirements.txt; if WordNet data is unavailable, the normalization lab prints a fallback vocabulary instead of downloading during class.

Lab 09: Sentiment Model

Course: TGS-2026064715 · **Maps to:** K8 K9 K10; A7 A8

Use stratified split, a leakage-safe pipeline, macro F1 and confusion matrix.

`hotel_reviews.csv` contains 120 fictional Singapore hotel and service reviews with dates, channel, rating, sentiment and property. Names and review text are synthetic; no customer records or external credentials are used.

1. Open a terminal in this lab folder. Confirm Python 3.10+ with `python3 --version`.

2. Install the shared requirements from the parent `labs/requirements.txt` with `python3 -m pip install -r ../requirements.txt`.

3. Inspect the first rows and column names in `hotel_reviews.csv`. Check that `review_id` is unique and `sentiment` has three classes.

4. Run `python3 run.py` from this folder. Read the printed output and any files created in `output/`.

5. Explain how the result supports the lab goal, what a production dataset would add, and one failure or bias risk.

© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

The script exits with code 0. Verify that the reported row count is 120. For model labs, inspect the holdout scores rather than training accuracy; for summaries, inspect the exported CSV/PNG.

How would different hotel review wording, label quality, or data leakage change this result?

Detailed procedure

33. Open lab-09-sentiment-model/hotel_reviews.csv and verify its columns, 120 synthetic rows, unique review IDs and sentiment labels.
34. From a terminal, change to labs/lab-09-sentiment-model and run python3 run.py.
35. Read the output, and inspect any generated files in labs/lab-09-sentiment-model/output/. Record the evidence in your notes.
36. Explain the result against the lab goal, then identify a sampling, label, preprocessing or modelling limitation.

Code walkthrough and interpretation

Read this lab-specific block from run.py before executing it. Identify input variables, transformations, printed evidence and output files. Change one parameter at a time, rerun, then explain the difference.

```
if MODE=='sentiment':

Xtr,Xte,ytr,yte=train_test_split(texts,df.sentiment,test_size=.25,stratify=df.sentiment,random_state=42
)

pipe=make_pipeline(TfidfVectorizer(stop_words='english',ngram_range=(1,2)),LogisticRegression(max_iter=
1000))
    pipe.fit(Xtr,ytr); pred=pipe.predict(Xte)
    print(classification_report(yte,pred,zero_division=0));
print('confusion',confusion_matrix(yte,pred,labels=pipe.classes_))

pd.DataFrame({'text':Xte,'truth':yte,'prediction':pred}).to_csv(OUT/'holdout_predictions.csv',index=Fal
se)
```

Record the exact command, observed output, interpretation and one limitation. If a package is missing, install from labs/requirements.txt; if WordNet data is unavailable, the normalization lab prints a fallback vocabulary instead of downloading during class.

Lab 10: Summarize Visualize

Course: TGS-2026064715 · **Maps to:** K11 K12; A9

Aggregate sentiment by property and service aspect; export a chart.

`hotel_reviews.csv` contains 120 fictional Singapore hotel and service reviews with dates, channel, rating, sentiment and property. Names and review text are synthetic; no customer records or external credentials are used.

1. Open a terminal in this lab folder. Confirm Python 3.10+ with `python3 --version`.

2. Install the shared requirements from the parent `labs/requirements.txt` with `python3 -m pip install -r ../requirements.txt`.

3. Inspect the first rows and column names in `hotel_reviews.csv`. Check that `review_id` is unique and `sentiment` has three classes.

© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

4. Run `python3 run.py` from this folder. Read the printed output and any files created in `output/`.

5. Explain how the result supports the lab goal, what a production dataset would add, and one failure or bias risk.

The script exits with code 0. Verify that the reported row count is 120. For model labs, inspect the holdout scores rather than training accuracy; for summaries, inspect the exported CSV/PNG.

How would different hotel review wording, label quality, or data leakage change this result?

Detailed procedure

37. Open lab-10-summarize-visualize/hotel_reviews.csv and verify its columns, 120 synthetic rows, unique review IDs and sentiment labels.
38. From a terminal, change to labs/lab-10-summarize-visualize and run python3 run.py.
39. Read the output, and inspect any generated files in labs/lab-10-summarize-visualize/output/. Record the evidence in your notes.
40. Explain the result against the lab goal, then identify a sampling, label, preprocessing or modelling limitation.

Code walkthrough and interpretation

Read this lab-specific block from run.py before executing it. Identify input variables, transformations, printed evidence and output files. Change one parameter at a time, rerun, then explain the difference.

```
if MODE=='summary':
    import matplotlib; matplotlib.use('Agg')
    import matplotlib.pyplot as plt

tab=pd.crosstab(df.hotel,df.sentiment,normalize='index').reindex(columns=['negative','neutral','positiv
e']).fillna(0)
    print(tab.round(2)); tab.to_csv(OUT/'sentiment_by_hotel.csv')
    ax=tab.plot.bar(stacked=True,color=['#EF6B6B','#A9B8C2','#16A6A0'],figsize=(9,5))
    ax.set_ylabel('Share of reviews'); ax.set_ylim(0,1); plt.tight_layout();
plt.savefig(OUT/'sentiment_by_hotel.png',dpi=150)
    aspects=['booking','room','breakfast','support','invoice','app']
    issues={a:int(((df.sentiment=='negative') & texts.str.contains(a,case=False)).sum()) for a in
aspects}
    print('negative aspects',issues)
```

Record the exact command, observed output, interpretation and one limitation. If a package is missing, install from labs/requirements.txt; if WordNet data is unavailable, the normalization lab prints a fallback vocabulary instead of downloading during class.

Assessment preparation

The WA covers K1–K12 from Topics 1–5. The PP uses one coherent hotel-review scenario and tests A1–A9. Practise the lab scripts, preserve screenshots or output CSVs, and cite the review IDs behind reported findings.

© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg